



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

课程报告

开课学期: 2026 夏季

课程名称: 计算机设计与实践

项目名称: 支持 miniRV 的五级流水线 Cache AXI SoC

项目类型: 综合设计型

课程学时: 48 地点:

学生班级: 待填写、待填写

学生学号: 2024311081、2024311453

学生姓名: 待填写、待填写

评阅教师: 待填写

报告成绩:

实验与创新实践教育中心制

2026 年 7 月 31 日

注: 本报告的各部分如果页数不够, 请同学们自行扩页。原则上一定要把报告写详细, 能说明清楚设计实现的主要逻辑、设计的成果、特色和过程等。设计报告将是评定每个人成绩的重要组成部分 (设计内容及报告写作都作为评分依据)。

设计概述			
组号	姓名	完成的指令	完成的其他内容
待填写	组员 A (2024311081, 姓名待填写)	RV32I 与 RV32M 指令共同验证	单周期/流水线数据通路、冒险处理、仿真及下板验收
	组员 B (2024311453, 姓名待填写)	RV32I 与 RV32M 指令共同验证	ICache/DCache、AXI、外设、C_TEST 与 CoreMark
设计的主要特色（除基本要求以外的设计）			
实现 RV32IM 五级流水线 CPU、AXI SoC、1 KiB ICache 与 1 KiB DCache；Cache 行宽 128 bit (4×32 bit)，直接映射，AXI 四拍整行回填；DCache 采用写直达、写不分配，MMIO 区域不缓存。实现 UART、LED、数码管和计时器等外设。Basic Trace 与 Cache AXI Trace 均通过 45/45；单周期 SoC 下板通过 C_TEST0~2；流水线 Cache AXI SoC 在 EGO1 上通过 CoreMark 正确性校验。			
流水线 SoC 在实现后（Post Implementation）的资源使用、功耗数据截图（要求不能有时序违例）			
最终 Cache 版本（50 MHz）：CoreMark=48.814，CoreMark/MHz=0.976，Iterations/Sec=50，700 次迭代用时 14 s；CRC 全部匹配并显示 FINISH。Vivado 实现后 WNS=0.986 ns、TNS=0 ns、失败端点=0/20810；LUT 30%、LUTRAM 9%、FF 15%、BRAM 98%、IO 29%、BUFG 9%、PLL 20%；片上总功耗 0.215 W、结温 26.0 °C、热裕量 59.0 °C。			

1 单周期 CPU 设计与实现

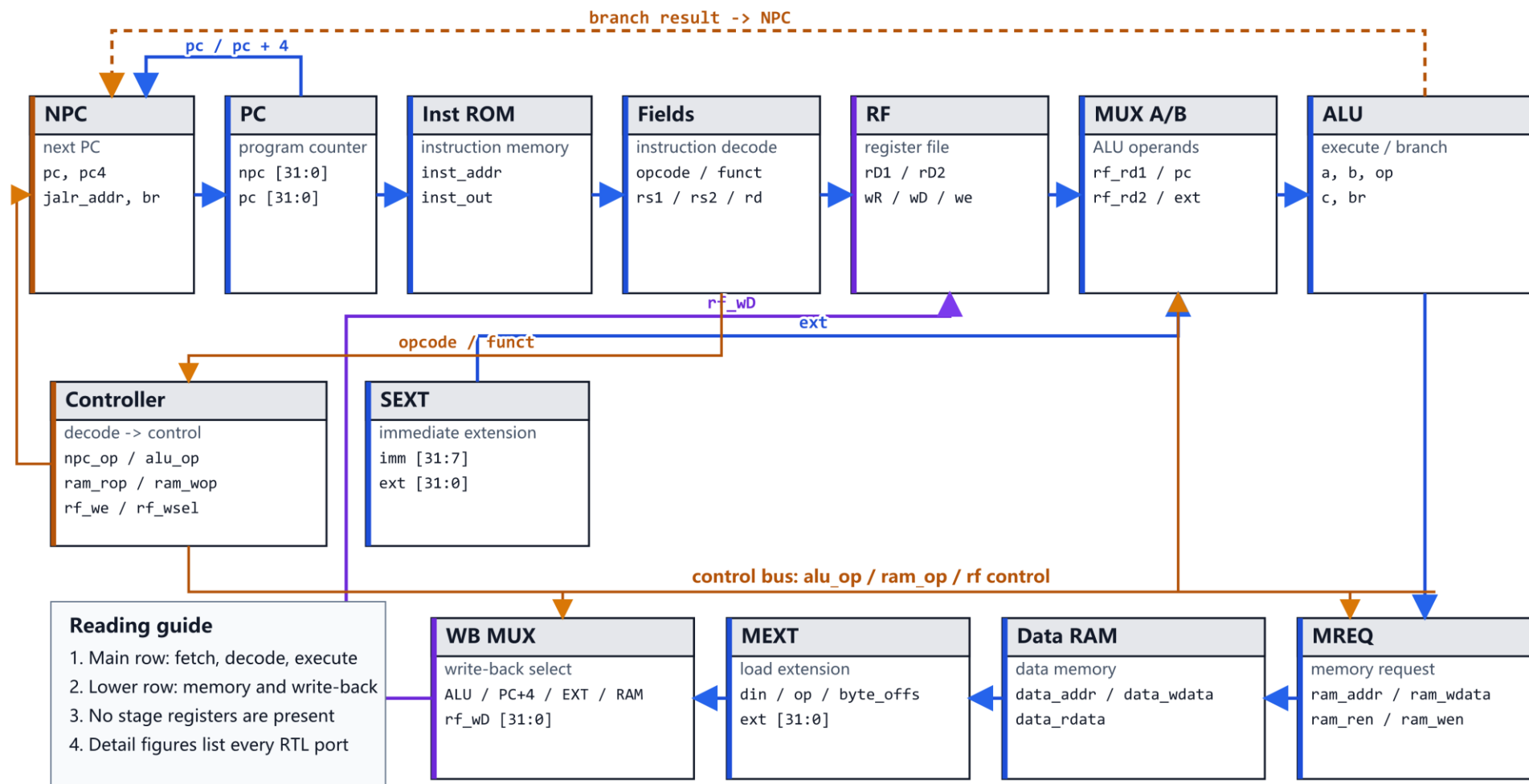
1.1 单周期 CPU 数据通路设计

要求：贴出**完整的**单周期数据通路图，无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，并用文字阐述各模块的功能。

完整的单周期数据通路总图见下一横向页；其后附模块级局部详图，便于核对接口信号名与位宽。

miniRV single-cycle CPU datapath

Report overview; IF / ID / EX / MEM / WB are logical regions, not pipeline registers.



Blue: 32-bit data | Brown: control / branch | Purple: register-file write-back

图 1-1 miniRV 普通单周期 CPU 数据通路总图（无流水级寄存器）

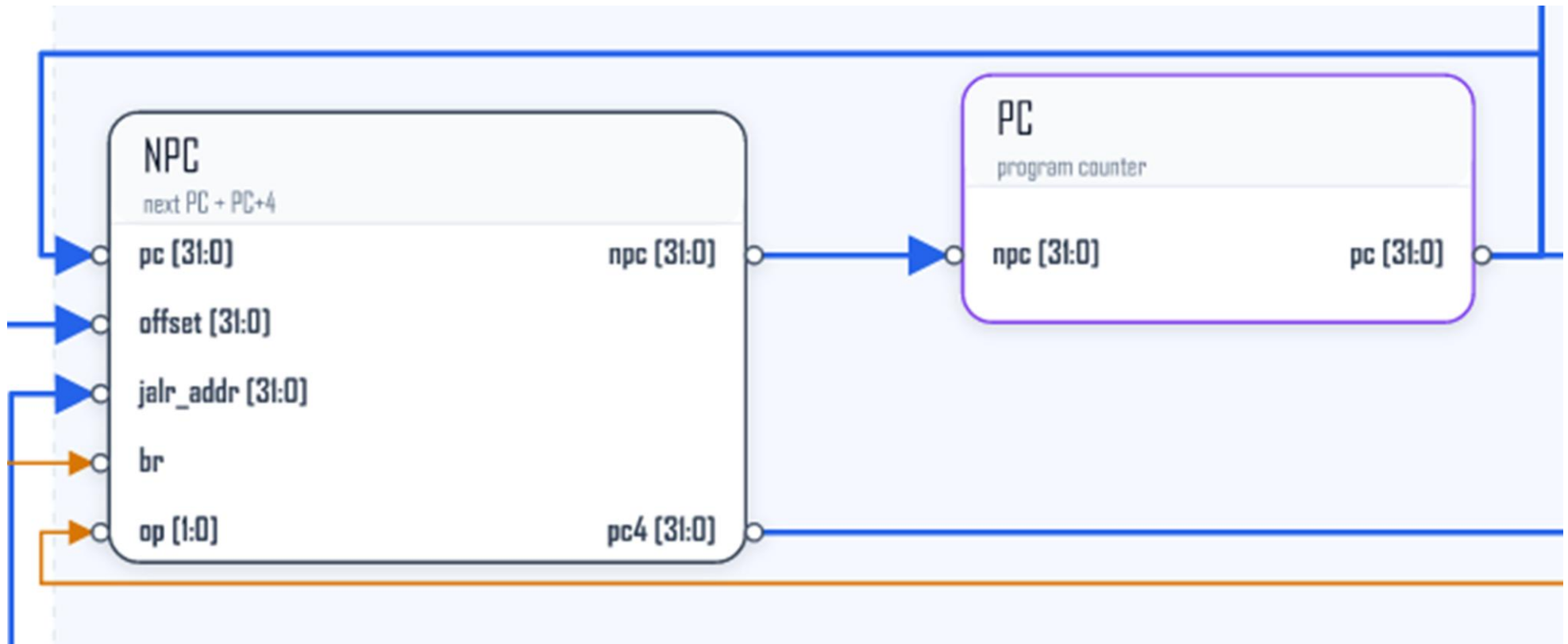


图 1-2 IF: 下一 PC 生成与 PC 寄存器

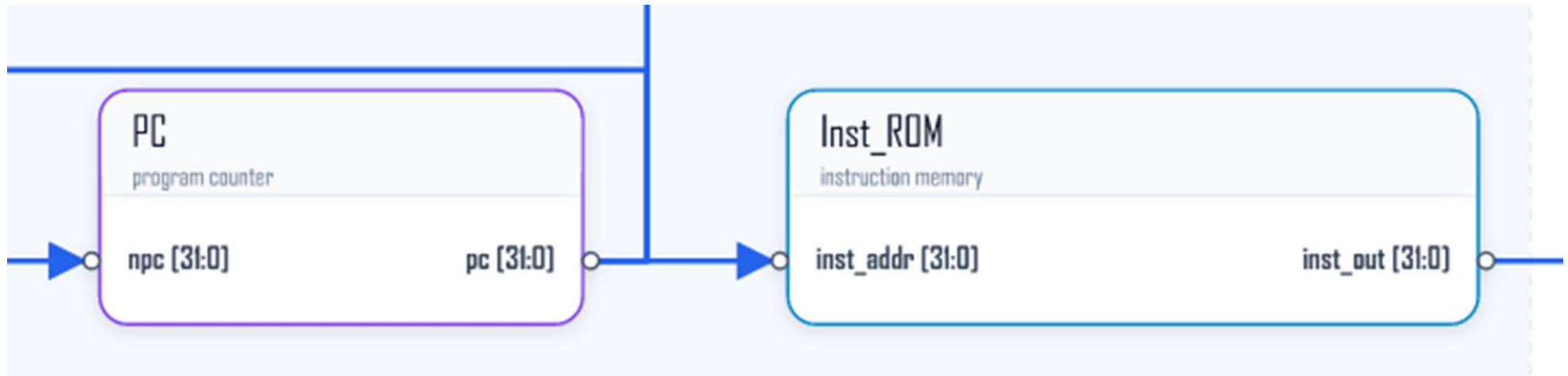


图 1-3 IF: PC 与指令存储器

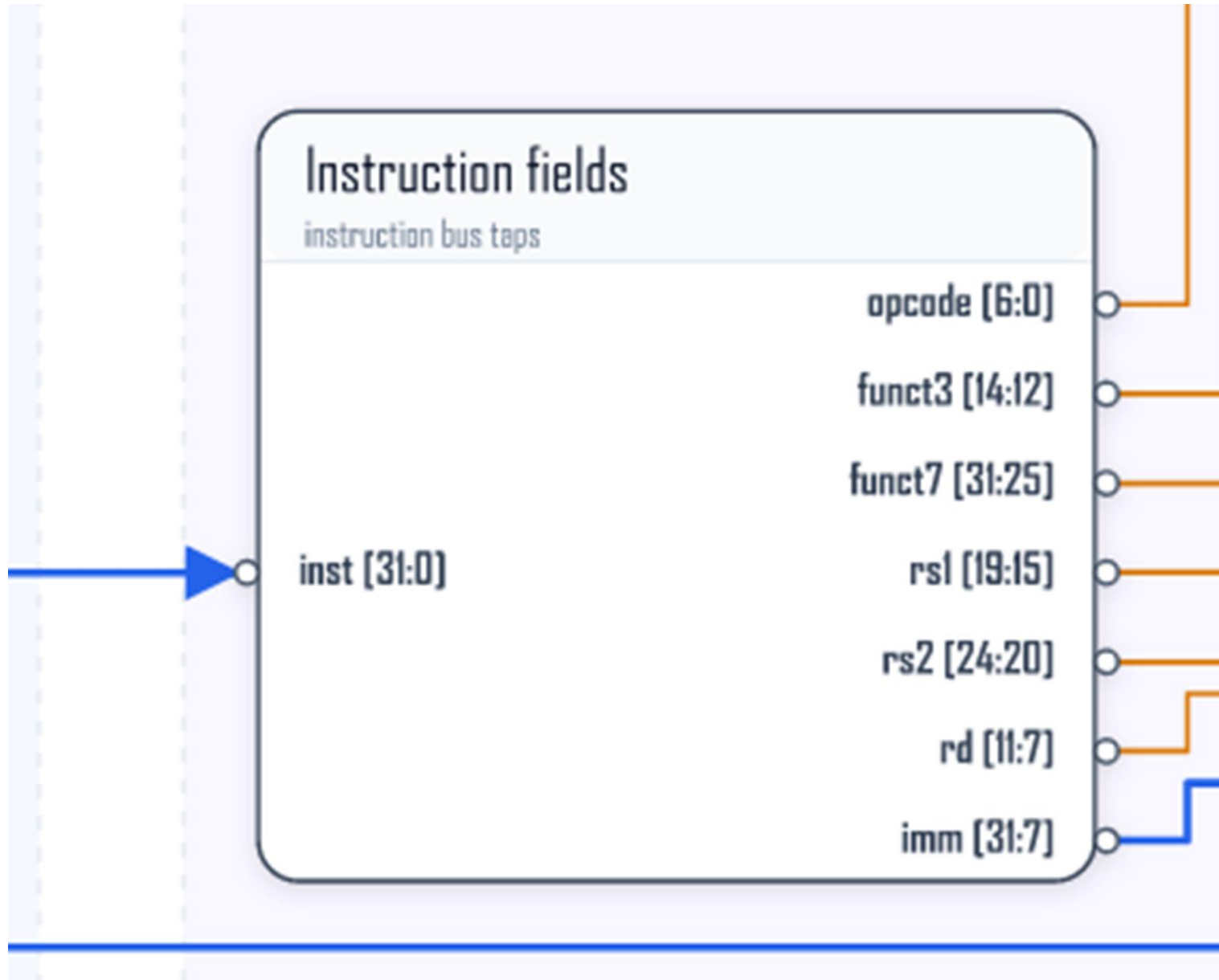


图 1-4 ID: 指令字段拆分

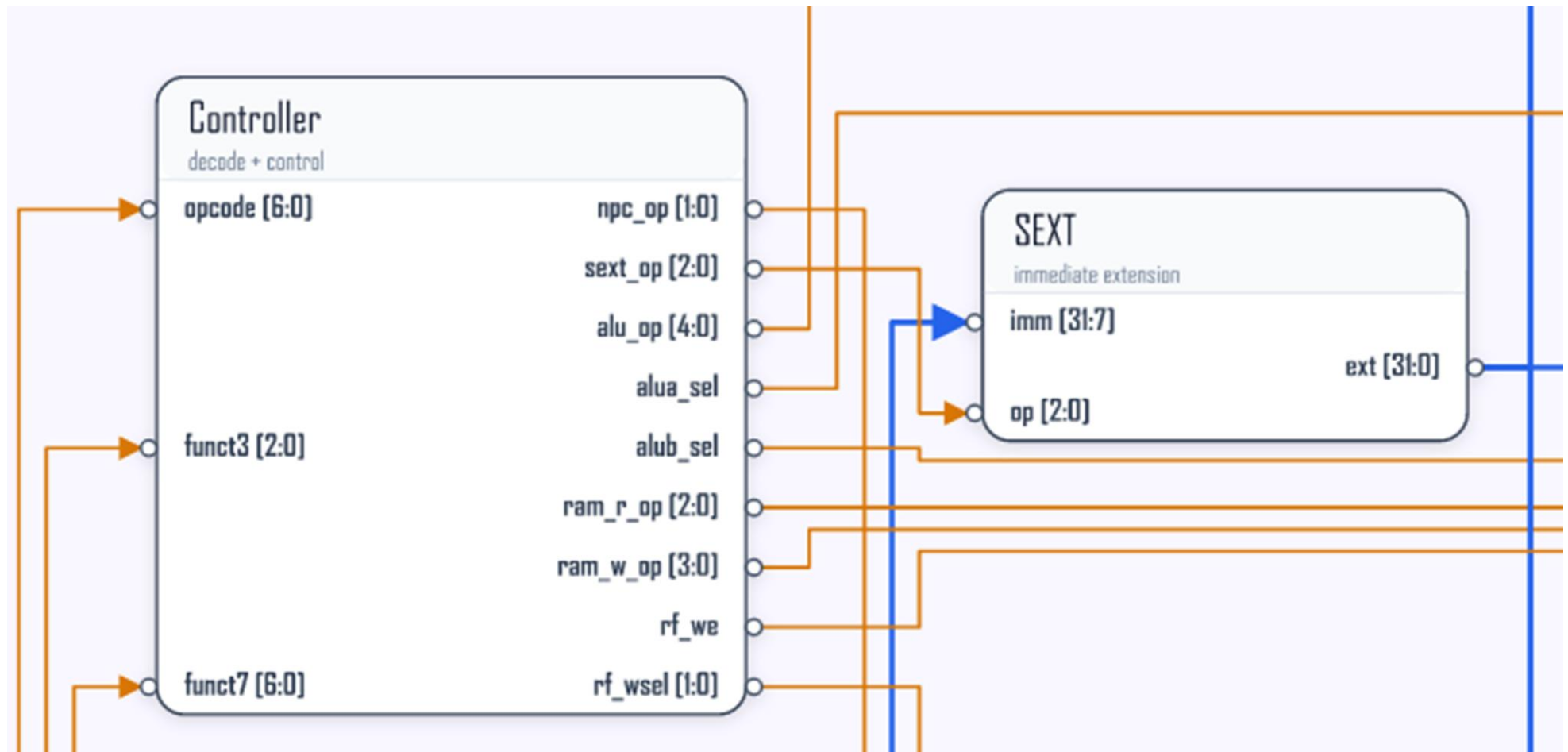


图 1-5 ID: 控制器与立即数扩展

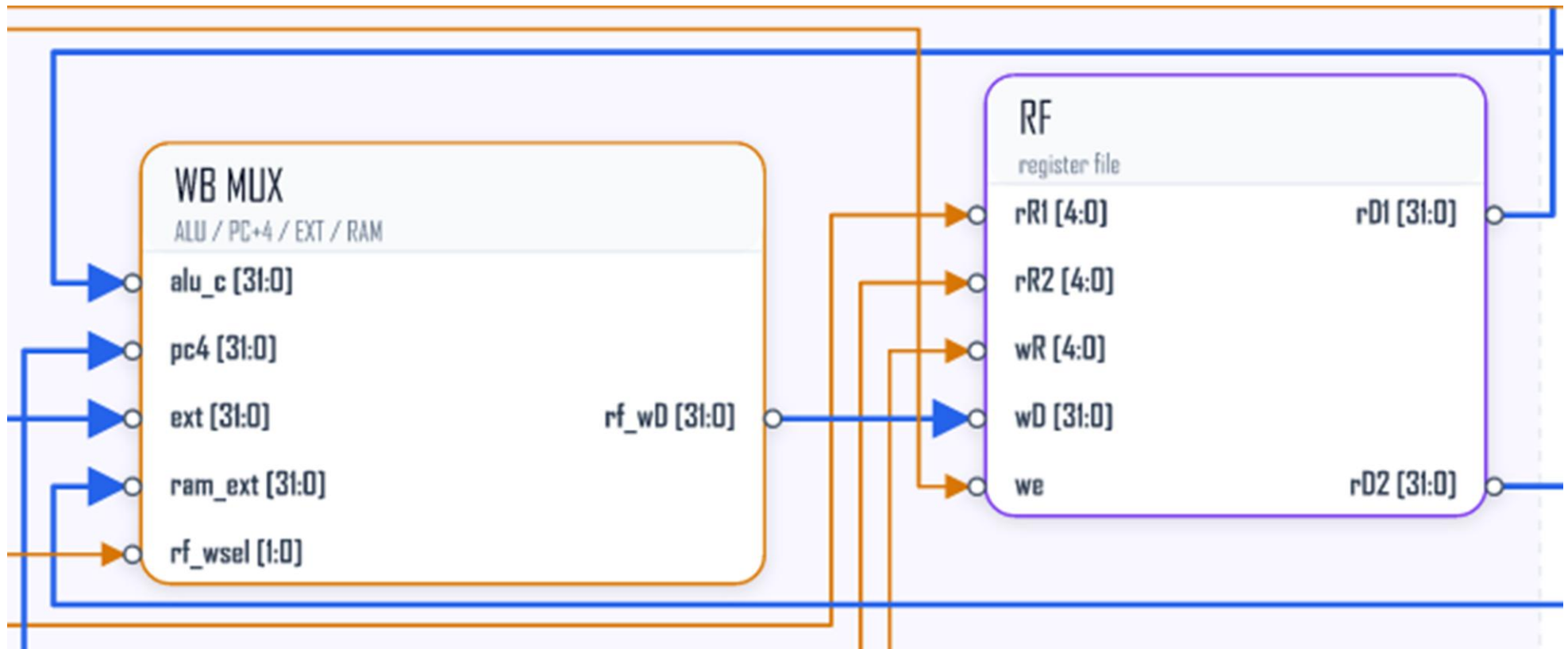


图 1-6 WB/ID: 写回选择与寄存器堆

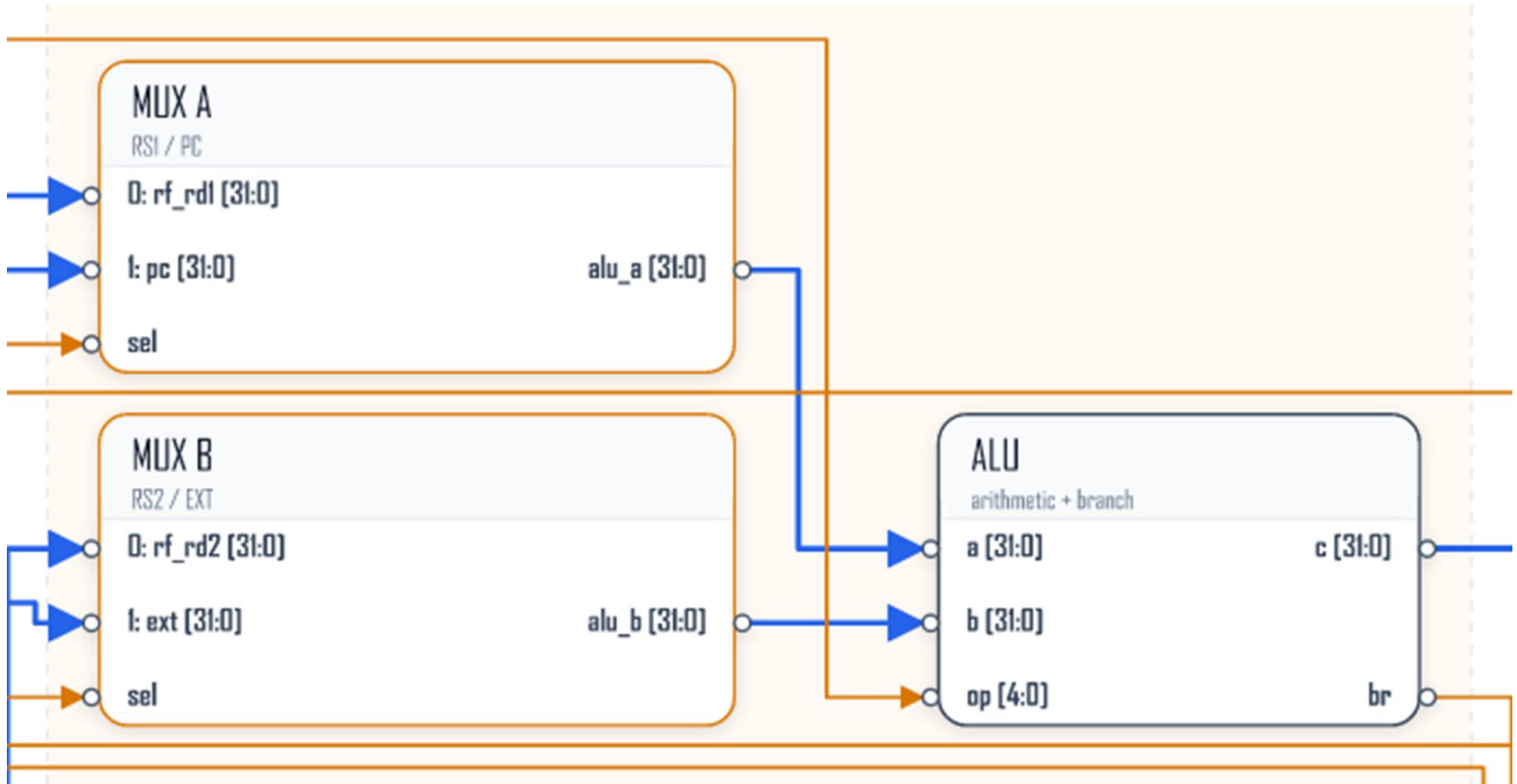


图 1-7 EX: 操作数选择与 ALU

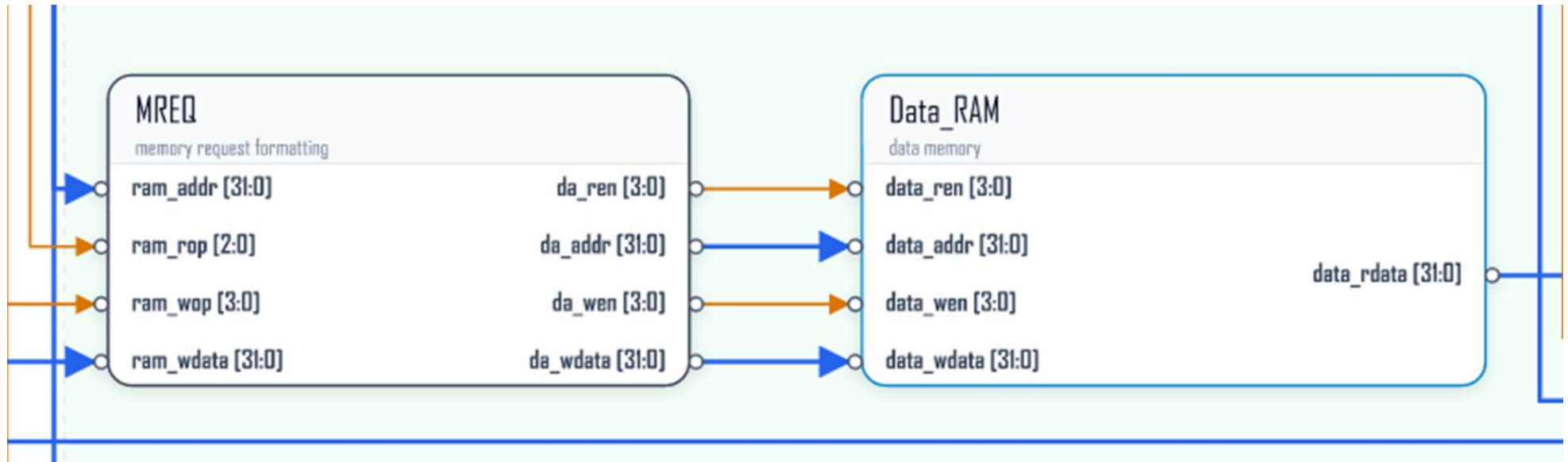


图 1-8 MEM:访存请求与数据存储器

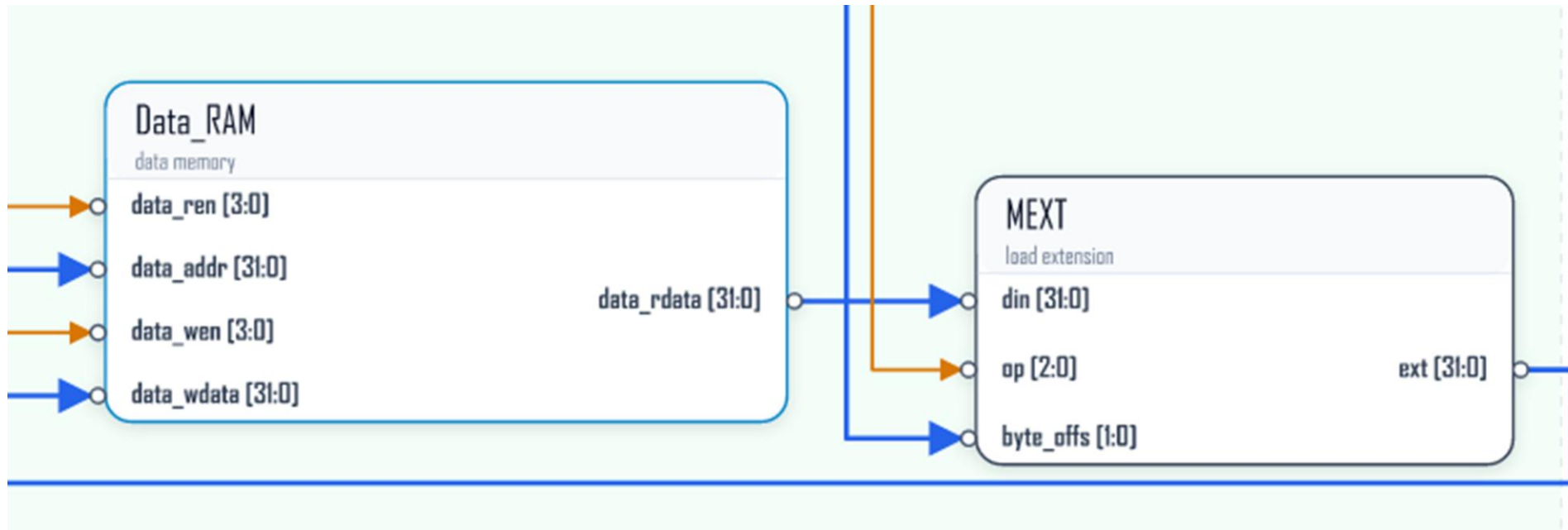


图 1-9 MEM/WB: 数据存储器和读数据扩展

1.2 单周期 CPU 详细设计

要求：以表格的形式列出各个部件的接口信号、位宽、功能描述等，并结合图、表、核心代码等形象化工具和手段，详细描述各个部件的关键实现。

1.2.1 总体实现

单周期 CPU 在一个时钟周期内完成取指、译码、执行、访存和写回。PC 给出指令地址, 控制器根据 opcode、funct3 与 funct7 产生控制信号; 立即数扩展器生成 I/S/B/U/J 型立即数, 寄存器堆提供两个源操作数; ALU 完成算术、逻辑、比较和地址计算, 访存结果或 ALU 结果经写回多路选择器写回 rd。所有组合路径在时钟上升沿前稳定, PC 和寄存器堆在上升沿更新。

部件	关键输入	关键输出	位宽	功能
NPC/PC	pc, imm, branch, jump	npc, pc	32	选择 PC+4、分支目标或跳转目标
IROM	pc	inst	32	按字地址读取指令
Controller	opcode/funct3/funct7	control bundle	1~5	生成 ALU、访存、分支与写回控制
SEXT	inst[31:7], sext_op	imm	32	生成并符号扩展 I/S/B/U/J 立即数
RF	rs1, rs2, rd, wd, we	rd1, rd2	5/32	双读单写通用寄存器堆, x0 恒为零
ALU	src_a, src_b, alu_op	result, zero/compare	32	算术逻辑、移位和比较
MREQ	addr, wdata, funct3	wen, byte_en, aligned_wdata	32/4	按字节/半字/字生成写掩码与对齐数据
MEXT	rdata, addr[1:0], funct3	load_data	32	lb/lbu/lh/lhu/lw 的截取与符号扩展
MULT/DIV	rs1, rs2, funct3	m_result	32/64	实现 RV32M 乘法与高半部分结果

1.2.2 控制与数据选择

ALU 第一个输入在 rs1 与 PC 之间选择, 第二个输入在 rs2 与立即数之间选择; 写

回数据在 ALU 结果、访存数据、PC+4 和乘除法结果之间选择。分支是否成立由 funct3 对应的有符号/无符号比较结果决定，JAL/JALR 在写回 PC+4 的同时修改下一 PC。

```
alu_src1 = alu_a_sel ? pc : rf_rdata1;
alu_src2 = alu_b_sel ? imm : rf_rdata2;
rf_wdata = wb_sel_mem ? load_data :
            wb_sel_pc4 ? (pc + 32'd4) :
            wb_sel_m   ? m_result : alu_result;
```

1.2.3 半字/字节访存

存储请求模块根据地址低两位产生字节使能：sb 仅使能一个字节，sh 使能相邻两个字节，sw 使能四个字节；写数据同时左移到对应字节位置。读回后，MEXT 按 funct3 选择 8、16 或 32 bit，对 lb/lh 做符号扩展，对 lbu/lhu 做零扩展。这样处理后，CPU 内部始终使用 32 bit 操作数，对齐细节被封装在访存边界中。

```
// 以 lh 为例：地址 bit[1] 选择高/低半字
half = addr[1] ? rdata[31:16] : rdata[15:0];
load_data = is_lhu ? {16'b0, half} : {{16{half[15]}}, half};
```

1.2.4 RV32M 实现

乘法先形成 64 bit 乘积，再按 MUL/MULH/MULHSU/MULHU 选择低 32 bit 或高 32 bit；除法处理被除数符号、除数为零以及 INT_MIN/-1 溢出等边界。单周期版本把 RV32M 作为组合计算路径，流水线版本则用 busy/done 与全局冻结协调多周期计算。

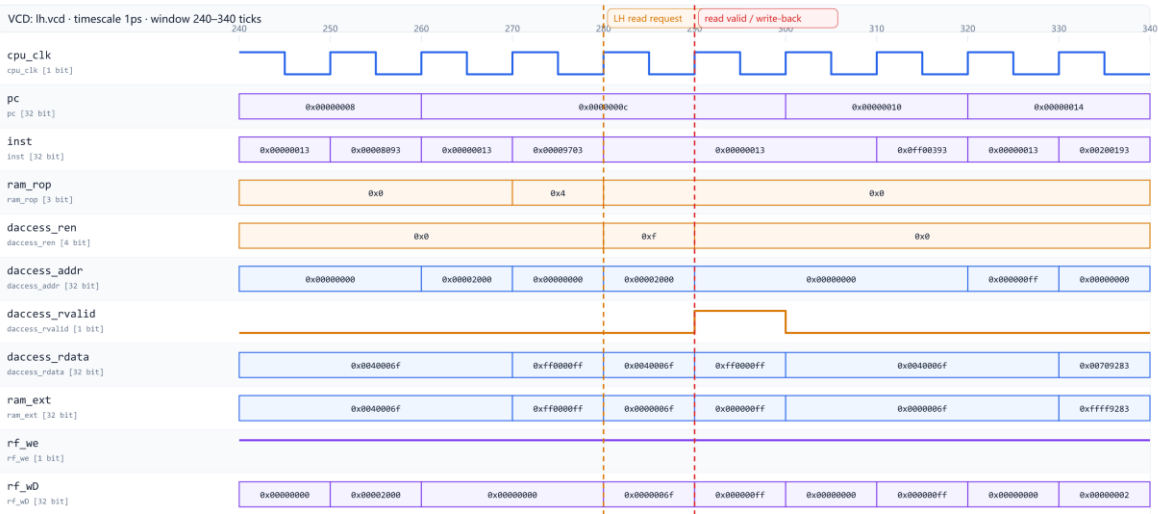
1.3 单周期 CPU 仿真分析

要求：在半字/字节访存指令、乘除法指令中分别任选 1 条指令，结合波形截图进行分析。截图需清晰，且需要包含信号名和指令执行所需的关键信号；此外，要结合波形具体时刻具体信号值来分析指令执行过程。

1.3.1 lh 半字加载波形

LH 半字加载：请求、返回与符号扩展波形

展示指令、访存读使能、地址、返回数据、MEXT 扩展结果与寄存器写回



蓝色：数据/普通控制 橙色：访存或多周期控制 红色虚线：关键事件，数值按十六进制显示。

图 1-10 lh 指令：地址计算、存储器返回与符号扩展

波形中，译码后 rs1 与 I 型立即数送入 ALU，得到有效地址；funct3=001 指示 lh。存储器返回 32 bit 对齐字后，地址低位选择目标半字，MEXT 取出 16 bit 并用其 bit[15] 扩展高 16 bit。写回使能只在该指令有效时产生，rd 获得最终 32 bit 符号扩展结果。该过程验证了地址计算、半字选择、符号扩展和写回通路的一致性。

1.3.2 mul 乘法波形

MUL 乘法：启动、忙等待、结果与写回波形

展示多周期乘法在 ALU 中的启动脉冲、busy 区间、结果产生和指令完成

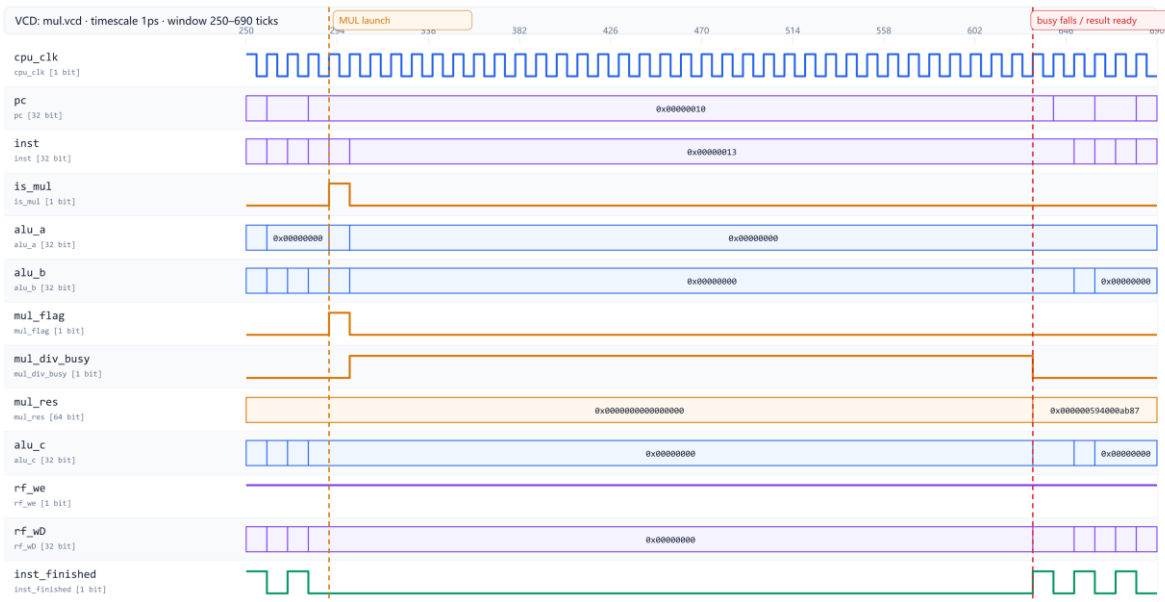


图 1-11 mul 指令：源操作数、64 bit 乘积与低 32 bit 写回

mul 的 opcode 为 OP、funct7=0000001。控制器选择 RV32M 运算，两个寄存器源操作数进入乘法器；64 bit 乘积形成后，MUL 选择低 32 bit 作为写回数据。波形中写回地址与 rd 一致，写回使能在指令完成时有效，说明 RV32M 结果选择未与普通 ALU 写回冲突。

1.3.3 自动测试结论

Basic Trace 对照参考提交逐条比较 PC、指令、寄存器写回地址和写回数据，最终 45/45 全部通过。因此除上述代表性指令外，其余 RV32I 指令和 RV32M 指令的控制、计算与写回也得到回归验证。

2 流水线 CPU 及 SoC 设计与实现

2.1 流水线 CPU 数据通路

要求：绘制完整的流水线数据通路图，无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，并用文字阐述各模块的功能。数据通路图应当能体现出流水线是如何划分的，并用文字阐述每个流水级具备什么功能、需要完成哪些操作。

2.1.1 五级划分与级间寄存器

流水线按 IF、ID、EX、MEM、WB 五级划分，IF/ID、ID/EX、EX/MEM、MEM/WB 四组级间寄存器分别保存数据、控制信号和 valid 位。valid 表明该槽位是否含有真实指令；插入气泡时清除 valid，冻结时保持对应寄存器不变，冲刷时同时使错误路径指令失效。

阶段	主要输入	主要工作	送往下一阶段
IF	PC、取指返回	产生取指地址并获得指令	PC、inst、valid
ID	inst、寄存器值	译码、读寄存器、生成立即数和控制	操作数、rd、imm、control
EX	ID/EX 数据	前递选择、ALU、分支、乘除法	ALU 结果、store data、rd、control
MEM	EX/MEM 数据	Cache/AXI/MMIO 访问	访存结果、ALU 结果、rd、control
WB	MEM/WB 数据	选择写回数据并提交 Trace	rf_we、rd、wdata

2.1.2 流水线控制原则

- 正常推进：每拍把当前级的 data/control/valid 一并送入下一级。
- 数据相关：优先用 EX/MEM、MEM/WB 前递；无法前递的 load-use 插入 1 个气泡。
- 控制相关：分支/跳转在 EX 确认后修改 PC，并冲刷更年轻的 IF/ID、ID/EX。
- 长延迟：Cache miss、AXI 等待和 M 扩展 busy 产生 effective_freeze，保持在途状态。
- 精确提交：只有 valid 且未冻结的 WB 指令才能写寄存器并产生 Trace。

miniRV five-stage pipelined CPU datapath

Block-level report view; exact register payloads and control conditions are listed below the drawing.

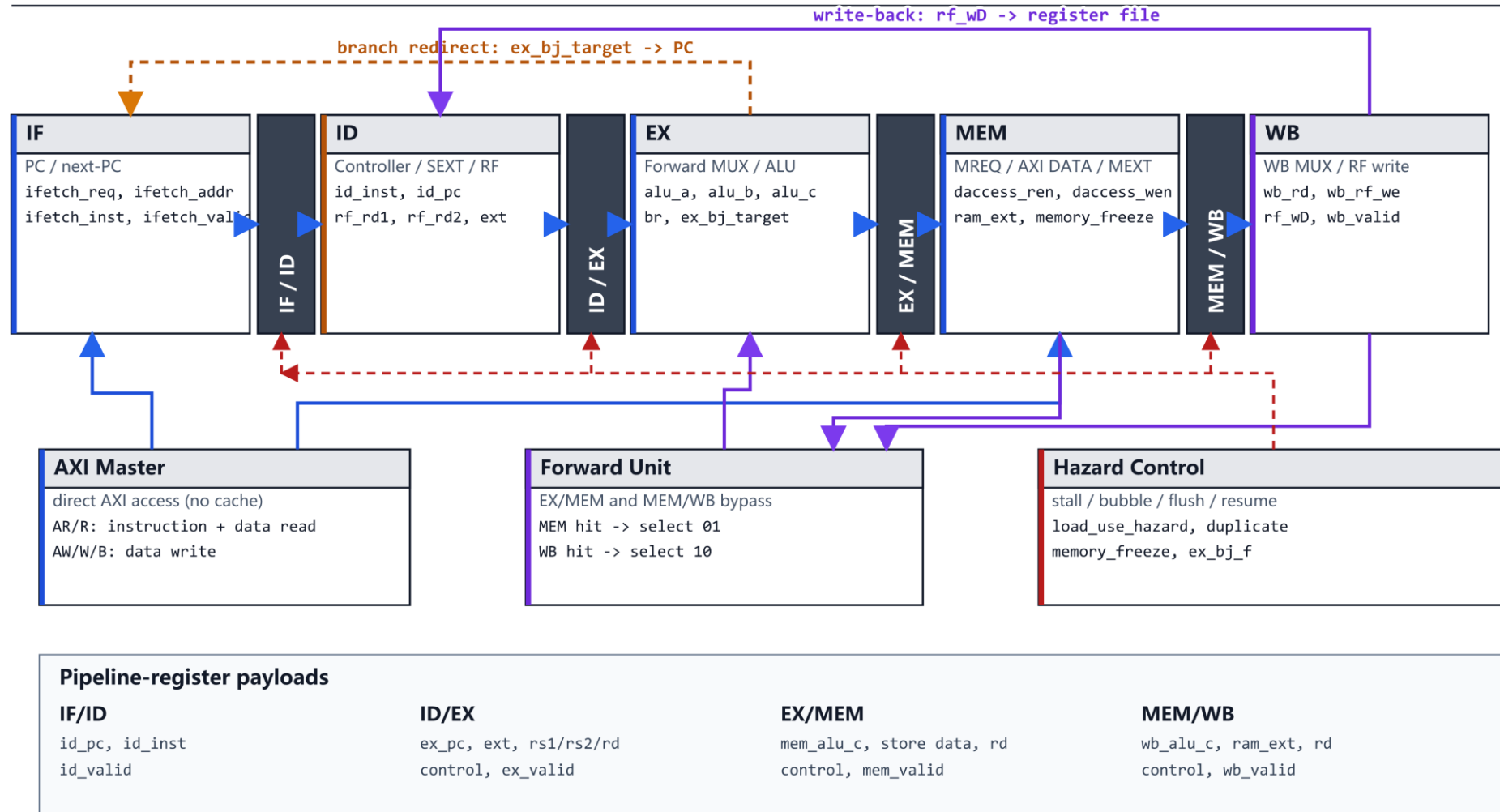


图 2-1 miniRV 五级流水线 CPU 数据通路总图

2.2 流水线 SoC 详细设计

要求：(1) 结合关键代码详细说明流水线冒险的检测及解决方法。(2) 绘制总线模块的状态转换图，标注清楚状态转移条件及每个状态产生的输出信号；结合关键代码详细说明总线及外设 I/O 接口电路的实现逻辑。

2.2.1 RAW 前递与 Load-Use 停顿

EX 阶段的两个源操作数分别比较其 rs1/rs2 与 EX/MEM、MEM/WB 的 rd。若上一条指令结果已经在 EX/MEM 可用,则优先从 EX/MEM 前递;否则从 MEM/WB 前递。load 的数据直到 MEM 访问完成才有效,因此当 ID 指令读取 EX 阶段 load 的 rd 时,冻结 PC 和 IF/ID, 同时把 ID/EX 的 valid 清零, 形成一个气泡。

```
load_use = id_valid && ex_valid && ex_mem_read && (ex_rd != 0) &&
            ((id_use_rs1 && id_rs1 == ex_rd) ||
             (id_use_rs2 && id_rs2 == ex_rd));
if (load_use) begin
    pc_en = 1'b0; if_id_en = 1'b0; id_ex_valid_next = 1'b0;
end
```

2.2.2 分支、跳转与多周期冻结

分支和跳转在 EX 得到最终目标地址与 taken 结果。taken 时优先执行 redirect: 更新 PC, 并清除 IF/ID 与 ID/EX 的 valid, 保证顺序路径上已取到但不应执行的两条年轻指令不会提交。乘除法、Cache miss 或 AXI 等待期间, effective_freeze 保持各级寄存器和 PC, 同时禁止同一条指令重复从 ID 进入 EX; 完成握手出现后再恢复推进。

```
effective_freeze = m_busy || icache_wait || dcache_wait || axi_wait;
wb_commit = mem_wb_valid && !effective_freeze;
debug_wb_rf_we = {4{wb_commit && rf_we}};
```

2.2.3 ICache 与 DCache

I/D Cache 均为 1 KiB、直接映射、64 行,每行 16 Byte(128 bit,4 个 32 bit word)。32 bit 地址分为 tag、index 和 block offset: offset 4 bit 选择行内字节, index 6 bit 选择 64 行, 其余 22 bit 为 tag。读命中时由 valid、tag 相等共同决定; miss 时把地址按 16 Byte 对齐, 通过 AXI 发起 ARLEN=3 的四拍突发读, 收齐 128 bit 后一次写入

data/tag/valid。

属性	ICache	DCache
容量/组织	1 KiB, 64×16 B, 直接映射	1 KiB, 64×16 B, 直接映射
读 miss	四拍 AXI 整行回填	四拍 AXI 整行回填
写策略	只读	写直达、写不分配
外设访问	不涉及	MMIO 地址绕过 Cache
CPU 返回	行内目标 instruction word	行内目标 load word/扩展后数据

2.2.4 AXI ready/valid 与状态机

AXI 每个通道都用 VALID/READY 独立握手。VALID 表示发送方已经给出稳定、有效的地址或数据，READY 表示接收方当前能够接收；只有二者在同一个时钟沿同时为 1，传输才真正发生。读通路依次完成 AR 地址握手和若干 R 数据握手；写通路允许 AW 地址与 W 数据独立握手，两者均完成后等待 B 响应。状态机只能在握手条件成立时推进，不能把 VALID 单独出现误当成传输完成。

```
ar_fire = M_AXI_ARVALID && M_AXI_ARREADY;
r_fire  = M_AXI_RVALID  && M_AXI_RREADY;
aw_fire = M_AXI_AWVALID && M_AXI_AWREADY;
w_fire  = M_AXI_WVALID  && M_AXI_WREADY;
b_fire  = M_AXI_BVALID  && M_AXI_BREADY;
```

2.2.5 外设连接与地址译码

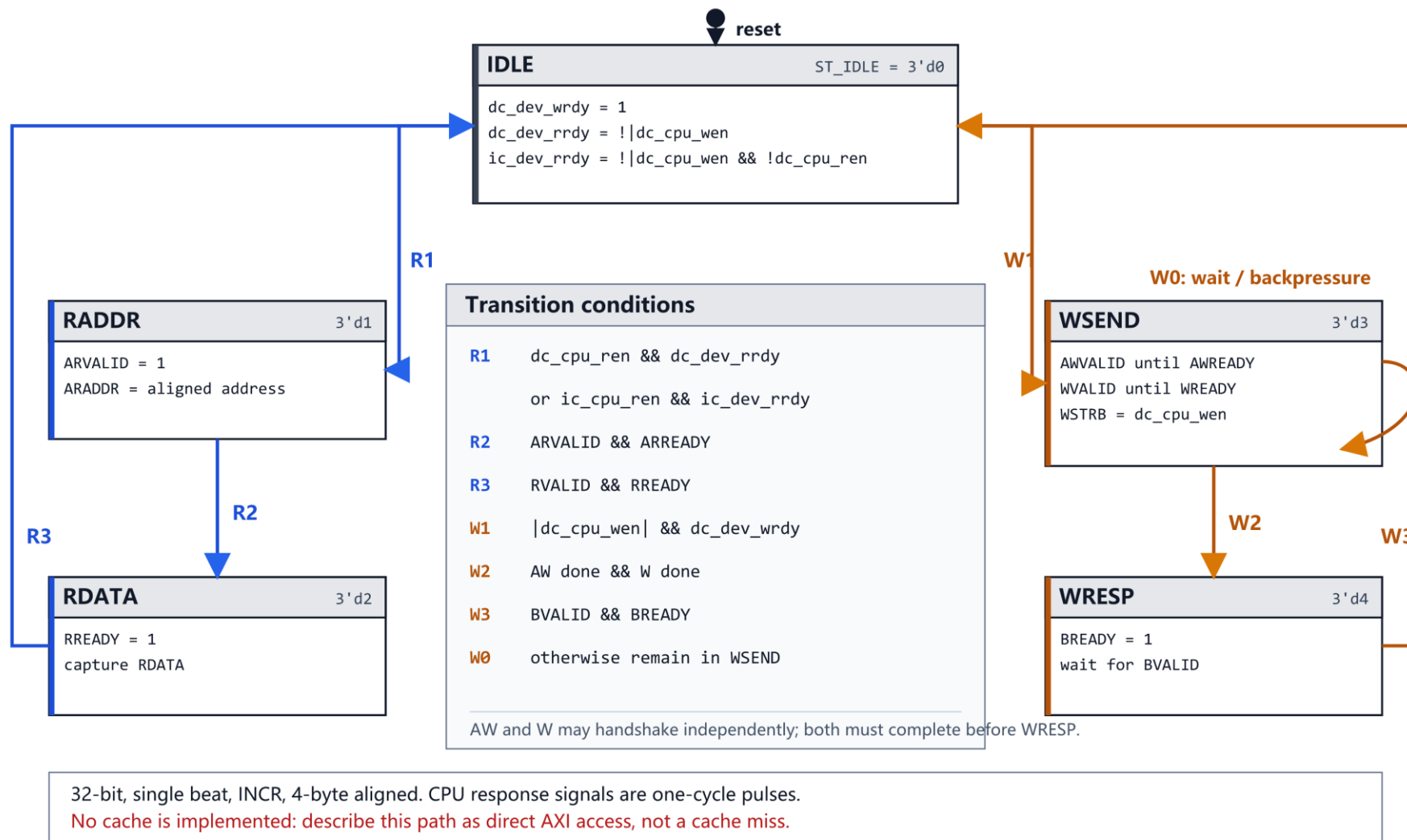
CPU 的数据访问先按地址空间译码：普通存储空间进入 DCache/AXI，MMIO 空间绕过 Cache，由 AXI 外设从机把写地址、写数据和字节选通转换为 UART、LED、数码管和计时器寄存器操作。UART 发送时 CPU 轮询状态寄存器，空闲后写 TX 数据寄存器；接收时 RX 有效置位，CPU 读数据寄存器取得字符。LED 与数码管是写寄存器外设，读写动作在 AXI AW/W/B 或 AR/R 握手中可直接观察。

连接层次	请求/响应	波形观察重点
CPU↔Cache	cpu_ren/wen、addr、rvalid、rdata	hit 时快速返回；miss 时保持请求
Cache↔AXI 主机	line req、line addr、line rvalid、	四拍拼接与整行提交

	128-bit line	
AXI 主机↔互连	AR/R、AW/W/B 五通道	VALID&&READY 握手、 RLAST/BRESP
互连↔外设	片选、寄存器地址、写数据、读数据	MMIO 不进入 DCache，副作用只 发生一次

AXI4 single-transaction master state machine (no cache)

RTL: axi_master.v | request priority: data write > data read > instruction read

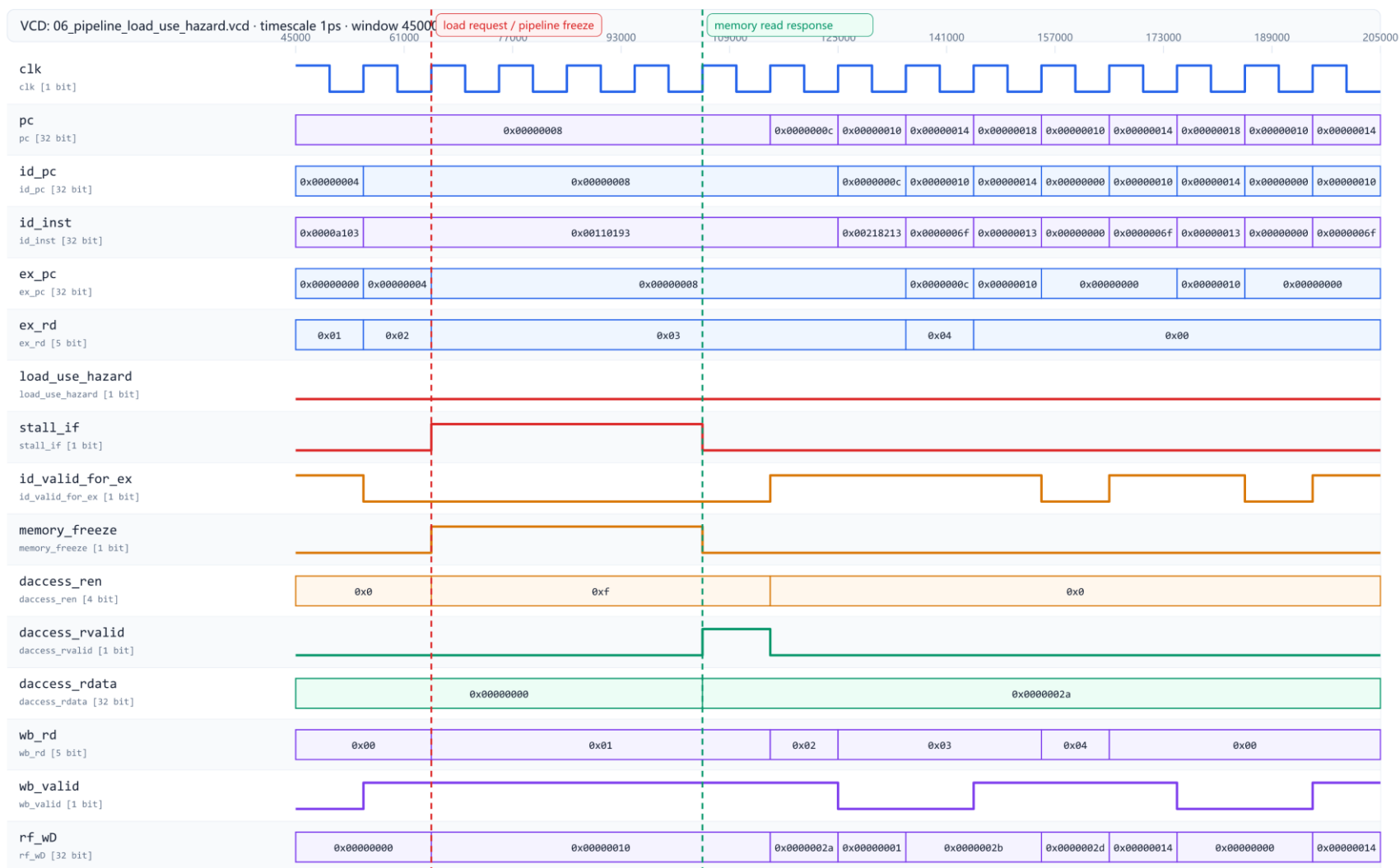


Blue: read states | Brown: write states | R1-R3/W0-W3 map arrows to the condition table.

图 2-2 无 Cache 的 AXI4 单事务主机状态转换图

流水线 Load-Use 测试：访存等待、气泡与恢复

程序序列为 LW x2,0(x1) → ADDI x3,x2,1; 本次波形由 memory_freeze 处理等待, load_use_hazard 保持低电平



蓝色：数据/普通控制 橙色：访存或多周期控制 红色虚线：关键事件。数值按十六进制显示。

图 2-3 流水线 Load-Use 测试中的访存等待、气泡与恢复波形

无 Cache 直接 AXI 读事务：AR/R 通道握手

同一波形中依次展示取指读取和数据读取；地址、VALID/READY 与返回数据均来自服务器 VCD

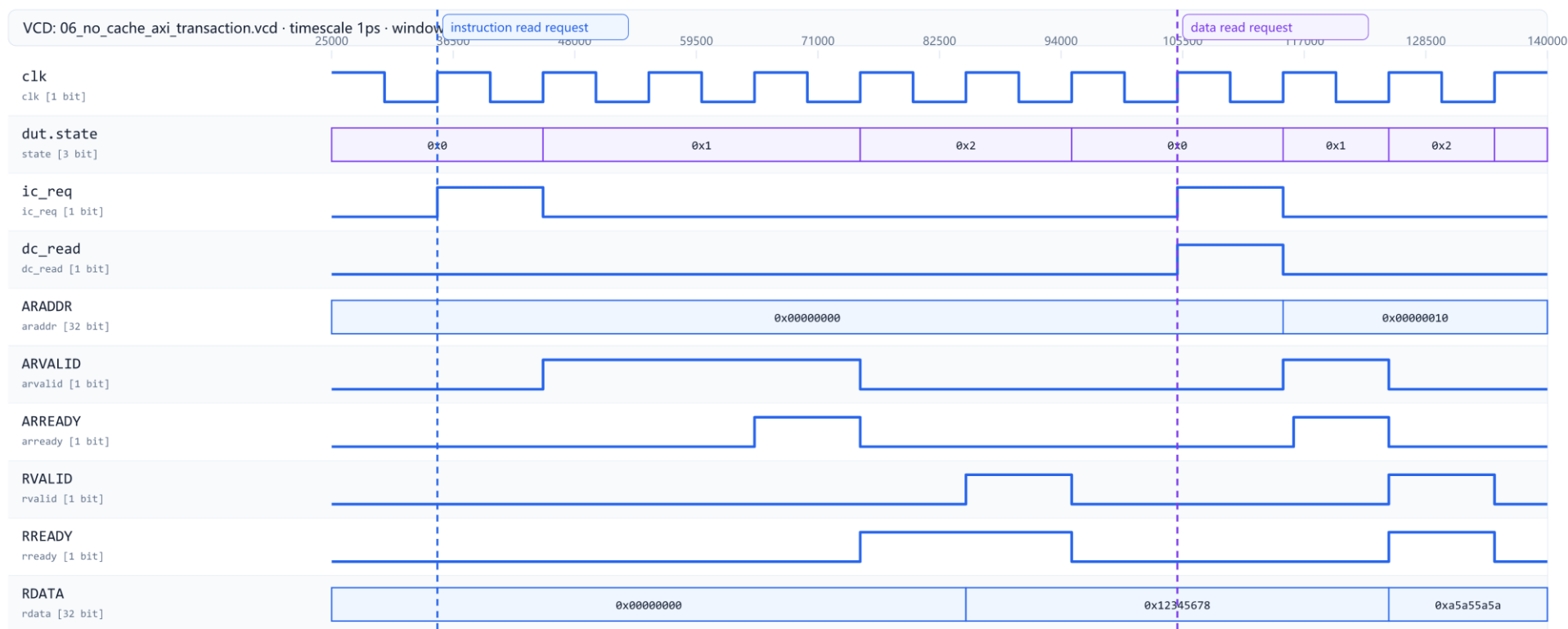
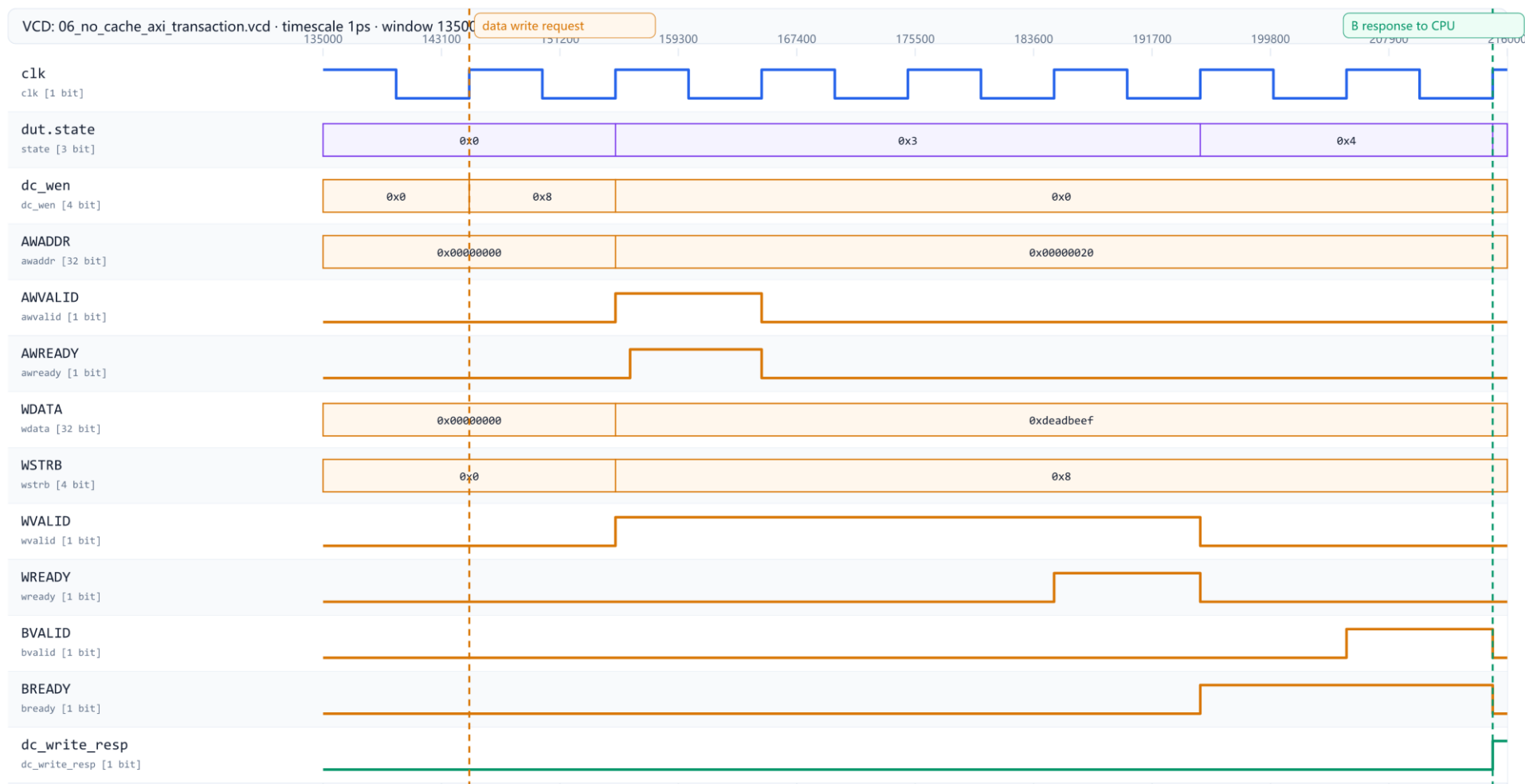


图 2-4 无 Cache 的 AXI 读事务及 AR/R 通道握手波形

无 Cache 直接 AXI 写事务：AW/W/B 通道握手

写地址和写数据通道可独立握手；两者完成后进入写响应阶段，并向 CPU 返回单周期响应



蓝色：数据/普通控制 橙色：访存或多周期控制 红色虚线：关键事件。数值按十六进制显示。

图 2-5 无 Cache 的 AXI 写事务及 AW/W/B 通道握手波形

2.3 流水线 SoC 仿真分析

要求：(1) 结合波形截图分析流水线 CPU 处理数据冒险的过程（若仅实现了理想流水，则结合波形截图分析理想流水执行指令的过程）。(2) 结合波形截图分析从 Cache 发生缺失并向总线发出访存请求，到总线读取主存并返回数据给 Cache 的过程。截图需清晰，且需要包含信号名和指令执行所需的关键信号；此外，要结合波形具体时刻具体信号值来分析指令执行过程。

2.3.1 Load-Use 冒险

图 2-3 对应一条 load 后紧跟使用其 rd 的指令。检测到相关后，PC 和 IF/ID 保持，ID/EX 的 valid 被清零；在数据返回前整条流水线因访存等待保持。load 数据进入 MEM/WB 后，相关指令恢复推进，并由 WB 前递得到正确源操作数。波形中应同时观察 id_rs1/id_rs2、ex_rd、ex_mem_read、load_use_stall、各级 valid、freeze 和写回数据，不能只看 PC 是否停住。

2.3.2 Cache miss、整行回填与随后命中

ICache miss、整行回填与随后命中

来源：10_cache_refill_hit_uncached.vcd；CPU 地址 0x24 所在行为 0x20 ~ 0x2F。

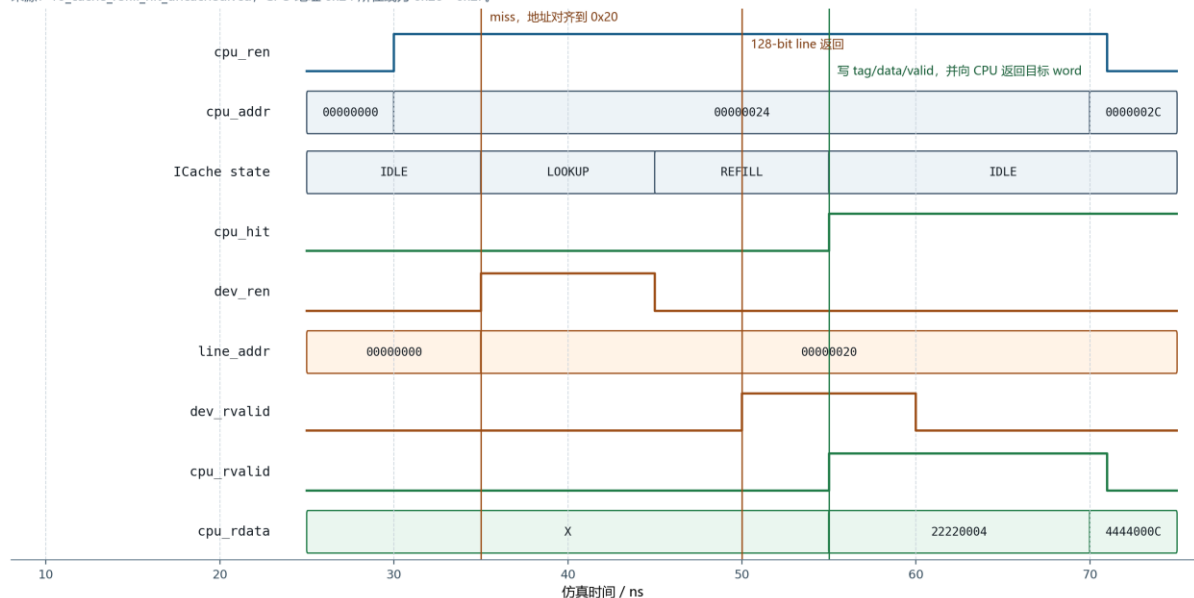


图 2-6 ICache miss 后 128 bit 整行回填并返回目标 word

CPU 请求地址 0x24，属于 0x20 ~ 0x2F 的同一 Cache 行。LOOKUP 判定 miss 后，line_addr 对齐到 0x20 并向下层发请求；下层返回 128 bit 行时写入 tag、data 和 valid，同

时从行内选择地址 0x24 对应的目标 word 返回 CPU。之后访问同一行内 0x2C 时无需再次发总线请求，命中后直接得到对应 word。该波形把“miss 会写入 Cache，随后同一行命中”完整体现出来。

AXI 四拍 Cache Line refill

ARLEN=3 表示 4 个 32-bit beat; 仅在 VALID && READY 同时为 1 的时钟沿完成传输。

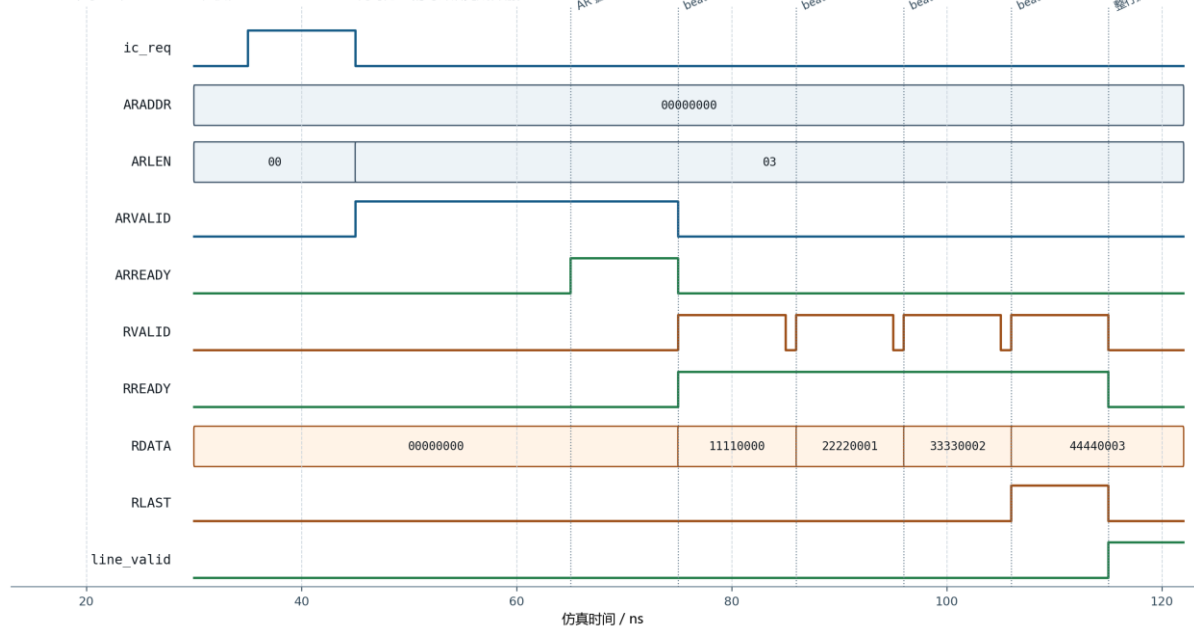


图 2-7 AXI ARLEN=3 四拍突发读与 Cache Line 提交

ARVALID 与 ARREADY 同拍为 1 时读地址被接受。ARLEN=3 表示总共传输 4 个 32 bit beat; 每次 RVALID 与 RREADY 同拍为 1 时, 当前 RDATA 才能拼入 line buffer。第四拍同时出现 RLAST, 主机在包含最后一拍数据后置 line_valid, 不能使用更新前的旧 buffer。图中四个数据拍依次拼成 128 bit Cache Line, 随后 Cache 才对 CPU 返回有效数据。

2.3.3 Trace 与 AXI 回归

测试	结果	主要覆盖
流水线 Basic Trace	45/45 PASS	五级推进、前递、load-use、分支冲刷、RV32IM 写回
流水线 Cache AXI Trace	45/45 PASS	I/D Cache hit/miss、突发读、写直达、长延迟精确提交
UART 输入/回显	PASS	RX 收到字符 A, 软件读出并回显, 确认主机到板卡方

		向有效
M 扩展板测	PASS	乘除法结果、busy/done 与流水线冻结/恢复

2.3.4 单周期 SoC: C_TEST0 ~ 2

三个程序均使用学号字符串 2024311081_2024311453 编译并更新 main.mem, 在 EGO1 重新生成 bitstream 后通过串口与板载外设完成验收。

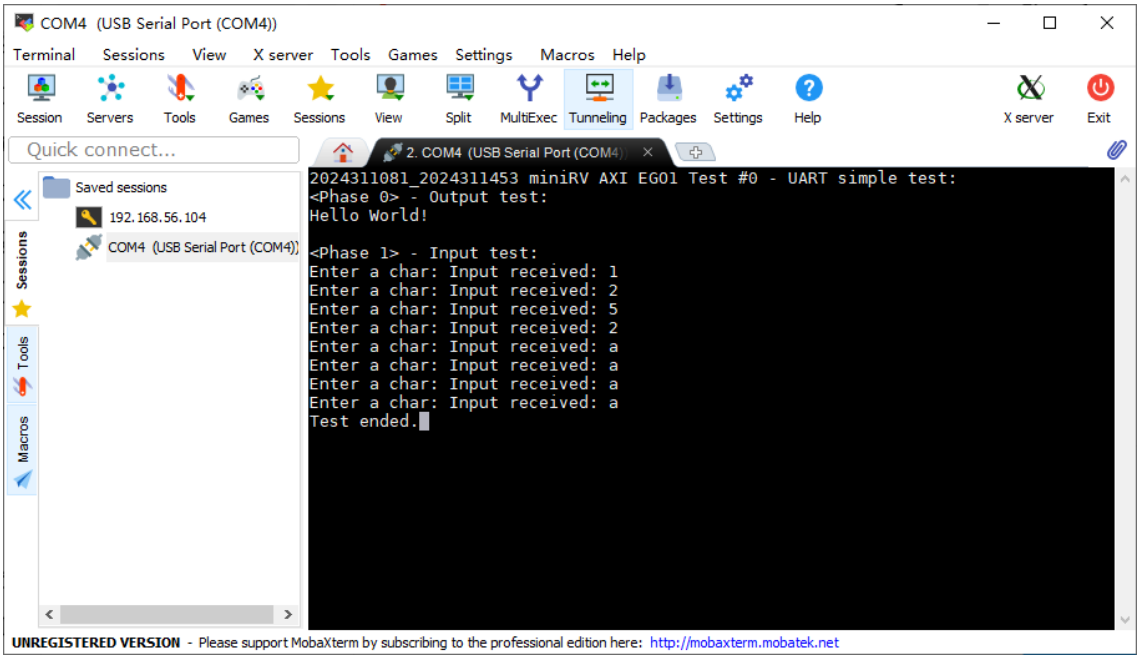


图 2-8 C_TEST0: UART 测试终端结果

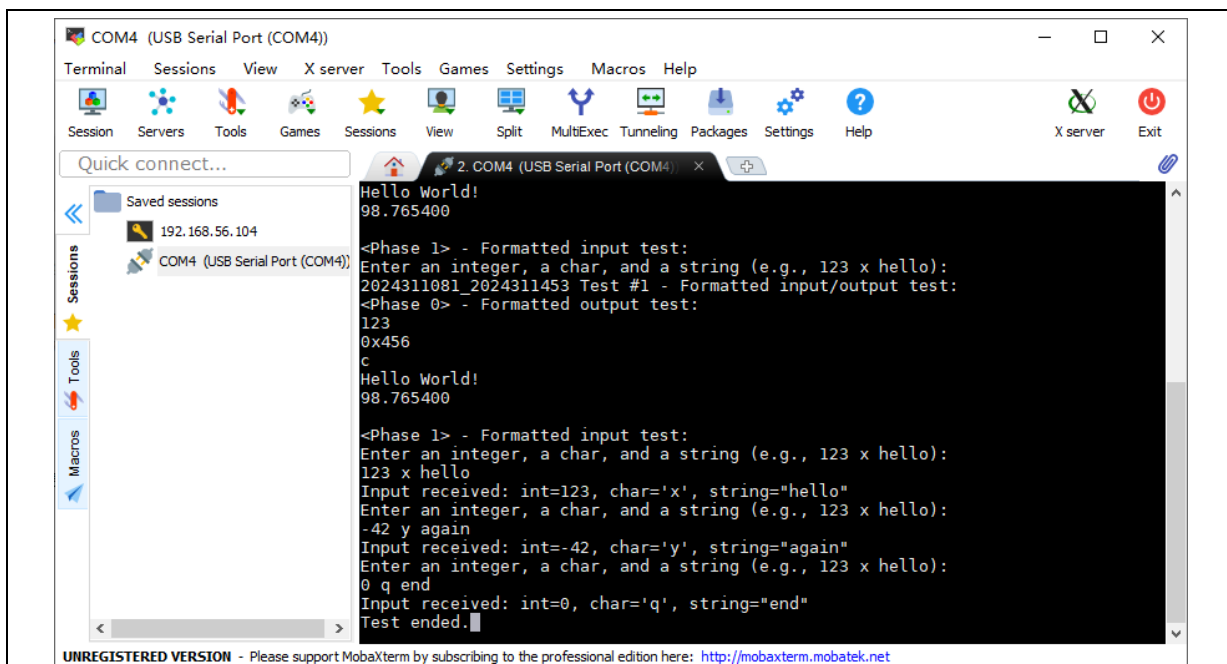


图 2-9 C_TEST1: 格式化输入输出测试终端结果

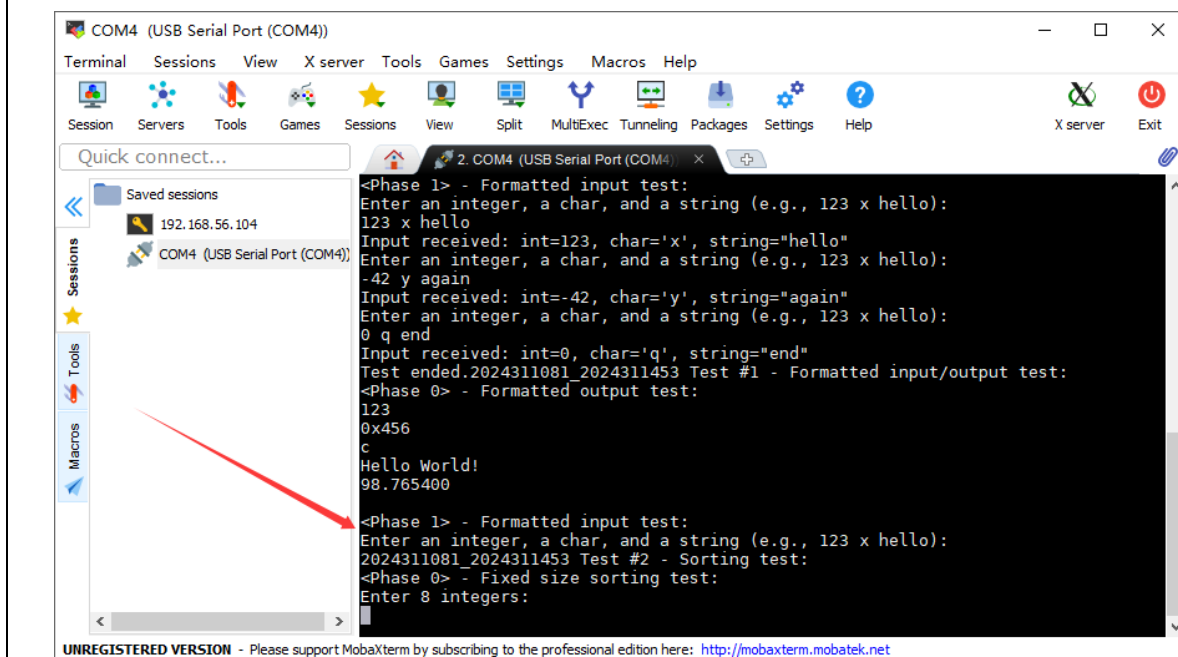


图 2-10 C_TEST2: 排序测试终端结果

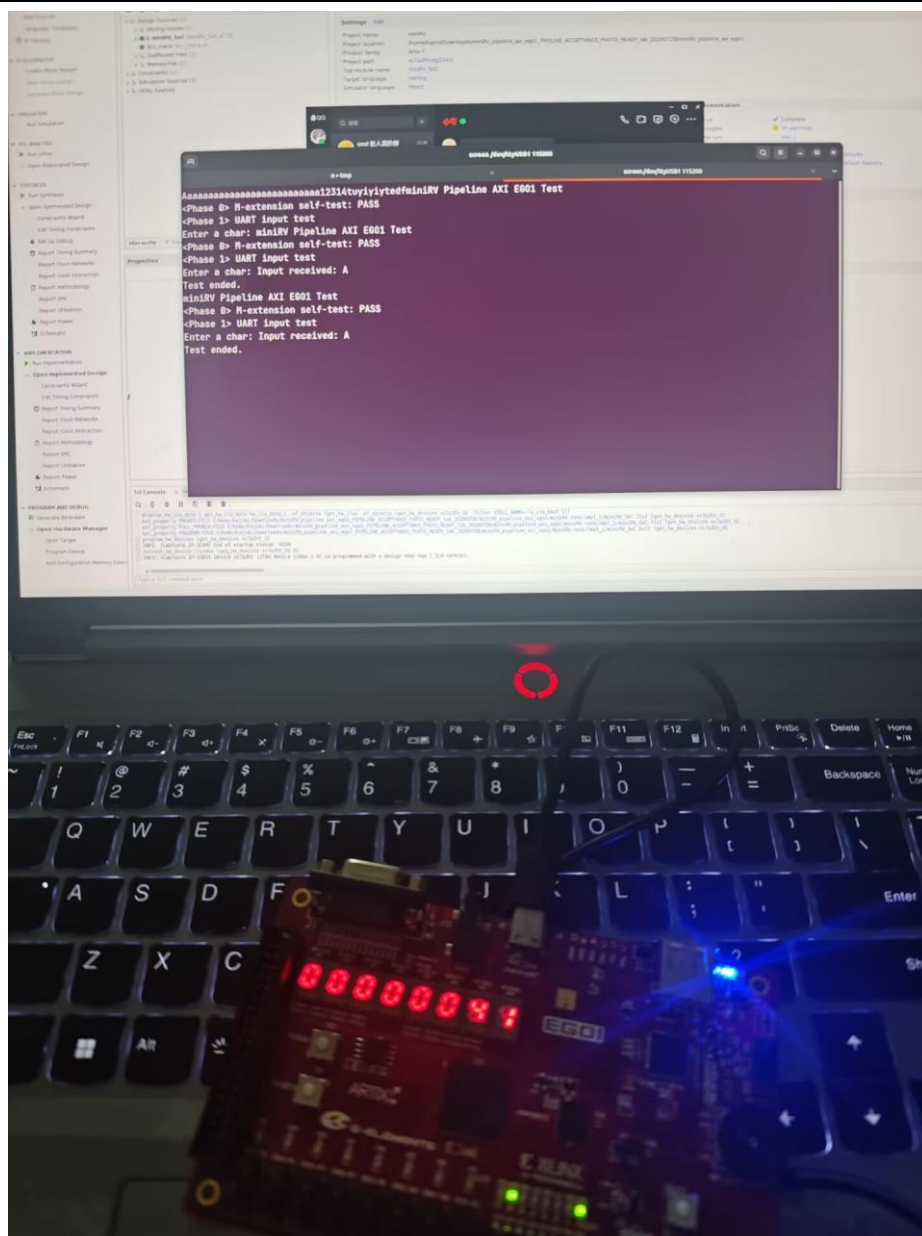


图 2-11 流水线 AXI SoC: UART 输入字符 A 并由软件处理

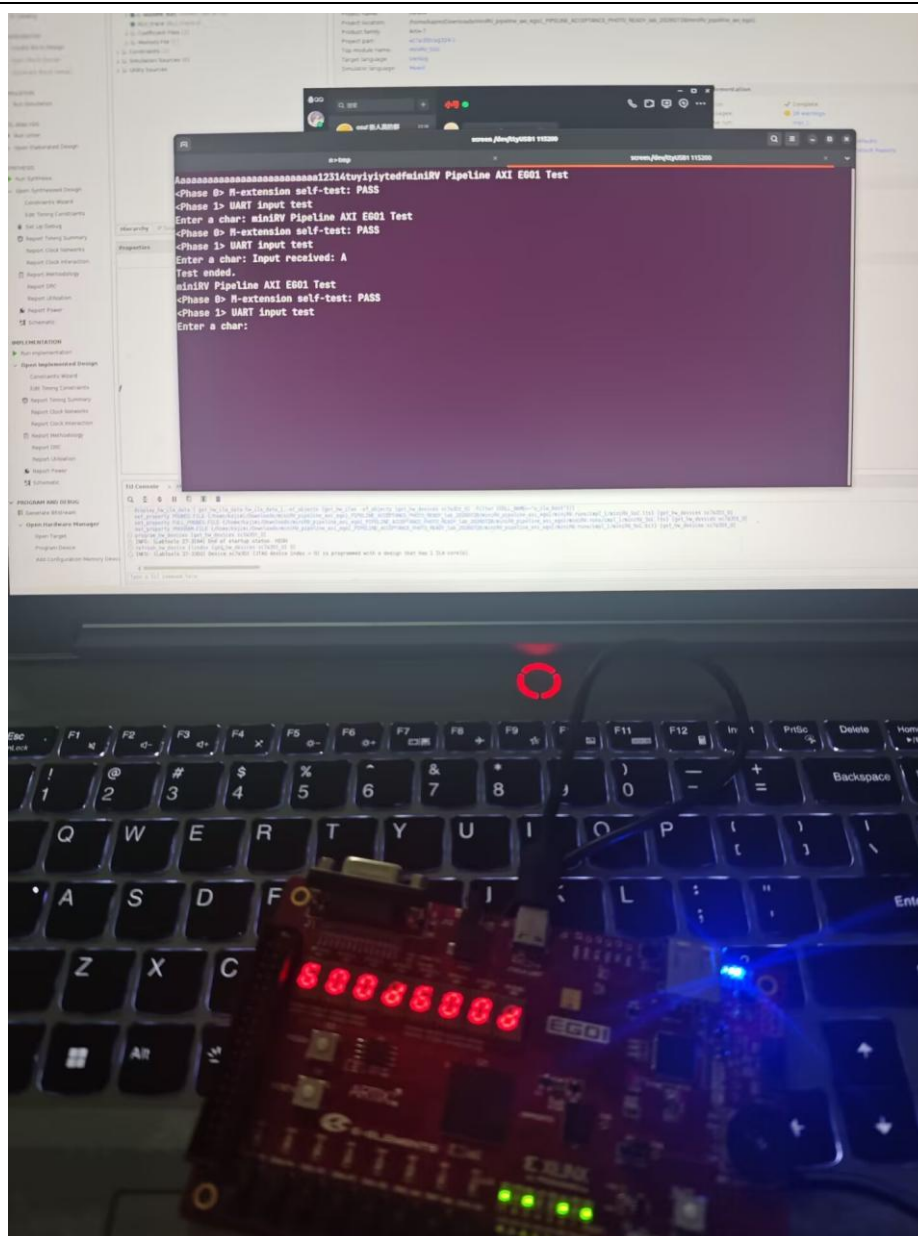


图 2-12 流水线 RV32M 板级测试通过

2.3.6 最终 Cache CoreMark 性能

最终验收截图显示：串口打印“miniRV Pipeline AXI EGO1 CoreMark”，学号为 2024311081_2024311453，CPU 主频 50 MHz。CoreMark 运行 700 次迭代，CRC 列表为 0xe9f5、0xe714、0x1fd7、0x8e3a、0x65c5，并明确输出“Correct operation validated”与“FINISH”，因此该数据满足正确性前提。

指标	无 Cache 基线	最终 Cache 版本	变化
----	------------	-------------	----

主频	50 MHz	50 MHz	相同
迭代次数	700	700	相同工作量
运行时间	32 s	14 s	缩短 56.3%
CoreMark	21.250	48.814	提升 2.30×
CoreMark/MHz	0.425	0.976	提升 2.30×
Iterations/Sec	21.875 (由 700/32 得)	50 (终端输出)	提升约 2.29×

性能提升的主要原因是程序和数据的局部性被 ICache/DCache 利用：命中访问不再为每个 32 bit word 重复经历 AXI 地址和数据握手，miss 时又通过四拍突发一次回填 16 Byte。因此总线固定开销被摊薄，CoreMark/MHz 从 0.425 提升到 0.976。

2.3.7 最终实现后资源、时序与功耗

类别	最终结果	判定
时序	WNS 0.986 ns; TNS 0 ns; 失败端点 0/20810	50 MHz 约束通过
资源	LUT 30%; LUTRAM 9%; FF 15%; BRAM 98%	逻辑资源充足，BRAM 接近上限
时钟/IO	IO 29%; BUFG 9%; PLL 20%	满足 EGO1 器件资源
功耗	片上功耗 0.215 W; 结温 26.0 °C	功耗与温度正常
热裕量	59.0 °C (12.3 W)	无热风险
实现状态	综合 Complete; 实现 Complete; DRC 5 warnings	无阻断性错误; 警告需随工程归档

BRAM 使用率达到 98%，这是最终版本最需要说明的资源约束：程序存储、数据存储以及分离式 I/D Cache 共同消耗块 RAM。虽然当前器件能够实现且时序通过，但若继续扩大 Cache 或加入更大程序镜像，需要减少片上存储深度、改用外部 DDR，或重新权衡 ICache/DCache 容量。

3 设计过程中遇到的问题及解决方法

要求：(1) 设计过程中遇到的有价值的错误，或测试过程中遇到的有价值的问题。所谓有价值，指的是解决该错误或问题后，能够学到新的知识和技巧，或加深对已有知识的理解和运用。(2) 组内讨论解决了哪些问题，至少列出 2 个。

问题 1：串口只收到板端发送，PC 输入无法驱动程序

最初现象是按 S6 能收到字符 U，说明板端 TX、USB 串口和波特率基本正常；但从终端输入 A 后程序无反应。仅凭终端是否本地回显不能判断字符是否发出，因此在 ILA 中加入 rx_data、AXI 读写地址/数据和 PC。波形确认 rx_data 长时间出现 0x41，说明字符已经进入 SoC，真正问题位于软件轮询/外设寄存器访问与流水线长延迟控制的衔接。通过对 MMIO 地址、状态位、读有效和重复执行路径逐级核对，最终恢复双向 UART。该问题训练了按“物理链路→UART RX→AXI/MMIO→CPU 软件”分层定位，而不是反复更换终端参数。

问题 2：长延迟冻结导致同一条指令重复进入 EX/WB

Cache miss、AXI 等待或乘除法 busy 时，若只冻结后级而没有阻止 ID 指令再次发射，同一条 MMIO 指令可能重复产生副作用，WB Trace 也可能重复提交。修正时统一定义 effective_freeze，冻结 PC 和所有在途级间寄存器，并用 valid 保证气泡不执行；debug_wb_rf_we 也只在有效、未冻结的提交周期产生。修正后 Basic Trace 与 Cache AXI Trace 均达到 45/45，UART 和外设写操作不再重复。

问题 3：Cache 行最后一拍丢失

第一次实现四拍回填时，在 RLAST 周期同时使用非阻塞赋值更新 line buffer 和提交整行，提交逻辑看到的是更新前的旧值，导致最后一个 word 缺失。解决方法是在最后一拍用“旧 buffer + 当前 RDATA”组合成 next_line，再写 data array 并拉高

line_valid。AXI 波形中第四拍必须同时满足 RVALID、RREADY、RLAST, 且 line_valid 只能在包含该拍数据后出现。

问题 4: 取指 miss 与分支重定向并发

当 ICache miss 正在返回时, 较老分支可能改变 PC。若不区分请求代次, 旧路径返回的指令会污染 IF/ID。设计中保留请求地址/代次信息, 在 redirect 后抑制旧返回, 清除年轻指令 valid, 并对新目标重新取指。现场观察时应同时查看 PC、ARADDR、取指返回地址、redirect/flush 和 IF/ID valid, 验证旧路径不会提交。

4 总结

要求：(1) 学完本课程后的个人收获；(2) 对本课程的建议和意见。请在认真总结和思考后填写。

4.1 课程收获

本项目从单周期 CPU 出发，逐步建立了五级流水线、冒险处理、AXI 总线、外设和 Cache，最终在 EGO1 上运行 C_TEST0~2 与 CoreMark。最大的收获不是某个模块能单独工作，而是理解了处理器、存储层次、总线协议和软件之间的因果链：一条指令从 PC 取址，经过译码与执行，在 Cache hit/miss、AXI ready/valid 和外设寄存器之间流动，最终以一次精确写回提交。

通过 Trace、VCD 与 ILA 三类证据，我们形成了分层调试方法。Trace 适合验证架构可见结果，VCD 适合解释流水级、冒险和 Cache 状态，ILA 适合确认板级复位、PC、AXI 与 UART 的真实交互。CoreMark 正确性 CRC、时序收敛和资源利用率共同说明最终结果既能运行，也具有可解释、可复现实证。

4.2 个人收获

- 组员 A (2024311081, 姓名待填写)：掌握 RV32IM 数据通路、五级流水线、前递/停顿/冲刷和波形阅读；最终分工文字请按实际情况微调。
- 组员 B (2024311453, 姓名待填写)：掌握 I/D Cache、AXI 五通道、MMIO 外设和下板调试；最终分工文字请按实际情况微调。

4.3 对课程的建议

建议课程在中期增加一次以“对着 RTL 和波形解释”为目标的模拟验收，并给出统一的 ILA 信号命名和 Cache/AXI 最小参考波形。这样可以让学生更早发现“能跑测试但讲

不清实现”的问题。同时建议明确报告中原始 VCD、下板照片、bit/rpt 与最终性能截图的归档清单，便于不同小组形成可复现证据链。

5 AI 大模型使用日志

要求：总结实验过程中使用 AI 大模型帮助你解决了哪些主要问题，分别给出提问的提示词并提炼 AI 大模型回答的主要内容。

本项目使用 AI 辅助梳理课程要求、解释 RTL、设计调试步骤和整理报告。AI 不替代功能验收；所有结论均由仓库代码、仿真回归、Vivado 报告、ILA/VCD 波形和 EGO1 实测交叉验证。

序号	提问提示词 (摘要)	AI 回答要点	人工验证与结果
1	更新 materials, 看课程验收和报告要求; 调整 README 目标。	按优秀目标拆分单周期 AXI、流水线 Basic/AXI、C_TEST0 ~ 2、流水线 CoreMark 和报告证据。	对照课程模板整理章节、图片和验收清单。
2	流水线怎么实现? load-use、前递、冲刷具体在代码哪里?	用五级 valid、EX/MEM 与 MEM/WB 前递、load-use 气泡、EX redirect 冲刷解释。	Basic Trace 45/45, 并用 hazard VCD 对照。
3	ready 和 valid 的区别是什么? 怎么看 AXI 波形?	VALID 由发送方保持, READY 由接收方给出; 只有同拍 VALID&&READY 才传输。	现场波形核对 AR/R、AW/W/B 握手与 RLAST。
4	ICache miss 会不会写入 Cache? Cache 多大、地址怎么分?	miss 时按行对齐, ARLEN=3 四拍回填 128 bit, 写 tag/data/valid; 1 KiB、64 行、16 B/行。	Cache refill 与 burst VCD 通过, Trace 45/45。
5	串口无输出、LED 全灭、数码管全零, 如何加入 ILA 排查?	按复位/时钟→PC/取指→AXI→MMIO/UART 分层加入信号, 区分 TX 正常与 RX 软件链路问题。	ILA 观察到 0x41, 最终 UART 双向和外设板测通过。
6	把已有图片、波形和最终性能数据写入课程实验报告。	保留教师模板; 把数据通路、关键代码、冒险、Cache/AXI、问题、总结和 AI 日志形成证据链。	生成 Word/PDF, 逐页检查; 最终数据来自验收后的最新截图。